CSE 490/590 Computer Architecture, Spring 2026
Project 1
Design of a 16-bit Processor (non-pipelined)

Task	**DUE DATE**
Group Registration	Saturday February 28, 2026 by 11:59 PM
Code Checkpoint	Saturday March 14, 2026 by 11:59 PM
Final Code and Report Submission	Saturday April 4, 2026 by 11:59 PM
Interview and Project Demonstration on Hardware	TBA (tentatively 8th April, 2026)

1 – Synopsis

This is a 6-week project to be done by group of 3 students. In this project, your team will implement, simulate, and synthesize a simple 16-bit processor.
To implement the processor you will use Xilinx Vivado and Verilog:

- Verilog is a Hardware Description Language (HDL), which is used to specify the structure and behavior of a hardware design.
- Vivado is an integrated development environment from Xilinx which includes many tools for the development, analysis, and synthesis to target hardware.

The Verilog code you write will implement the features of the 16-bit processor described in this document. Key tasks include:

1. Write correct Verilog code for different blocks and test them
2. Integrate these components into a complete (top level) design
3. Synthesize and run a software simulation to verify functionality
4. Create "bitstream" to target a Xilinx FPGA board, and program the FPGA board to verify the functionality of your design on real hardware.

2 – Tools and Resources

For information about how to install and get started with Vivado, see attached appendices

Appendices

- **Appendix 1:** includes instructions on how to install and use Vivado locally and remotely
- **Appendix 2:** includes instructions on how to create a project in Vivado
- **Appendix 3:** includes instructions on how to program the Basys 3 FPGA board.

For information about the Xilinx Basys 3 FPGA Board:
https://digilent.com/reference/_media/reference/programmable-logic/basys-3/basys3_rm.pdf

4 – Deliverables & Grading Score [100]:

This project has following deliverables (see table on front page for due dates):

1. **Group registration [5]**
2. **Code Checkpoint [10]**
3. **Final Source Code for working simulation [30]**
   This is your Verilog code of your 16-bit processor containing all components and top-level
4. **Updated working code used for hardware verification [15]**
   This is the final / updated code used for hardware verification and demonstration. It includes FPGA bitstream, flashing it onto FPGA, and showing results
5. **Final Project Report [20]**
   This is the final project report including diagrams, descriptions, and simulation results
6. **Live Interview and Hardware Demonstration [20]**
   Each member will be assessed individually. You will be asked questions about your work, and you will demonstrate the functionality of processor in simulation and on the FPGA.

4.1 – Group Registration:

This is a group project. Each group should have three students. Approach course staff if you are unable to find groupmates, or have less than three students. Register your group through this link before 2/28/2026 11:59 PM: Project Team Registration (CSE4/590) Spring2026 – Fill out form
Only one entry per group is required. Form will be closed after due date of registration.

4.2 – Code Checkpoint:

You should start the project by creating a shared repository, that will contain all the project data. This repository could be a GitHub or UBBox repository AND should be shared with Teaching Assistants so we can track the progress, and resolve issues if any. There is a deadline for submission of Code Checkpoint (see table above). You will upload a short report, **as a PDF**, documenting the work you have completed so far (e.g. descriptions of what components have been worked on). There must also be a brief section outlining the individual work assignment (division of tasks) and contributions of each member. We suggest you maintain following structure in your repository:

1. **WorkDivision** (.xls or .doc etc.): this file documents agreed upon division of work and the pledge that everyone will contribute to their best abilities
2. **WeeklyLog** (.xls or .doc etc.): update this file to show what you have been doing. It can include your meeting details, and work progress.
3. **SourceCode**: a subdirectory containing all implementation code
4. **ProjectReport**: you will produce checkpoint report and the final project report

You are expected to know how to work in groups and how to write reports. This code checkpoint will **not** be harshly graded; we are simply looking for evidence of some progress from your team.

4.2 – Verilog Source Code for Working Simulation:

You will use Verilog to describe the structure and behavior of your 16-bit processor. The individual components of the processor (see the list of required components in the processor specifications section) should be designed using **behavioral Verilog** (you may also use structural Verilog as well, but it is not recommended)**.** The processor as a whole (how all the individual components are connected together) **must** be designed with **structural Verilog.**

4.3 – Updated Verilog Code for Working Hardware Verification:

Once you have your design and simulation completed, you will then generate a bitstream and download it to an FPGA board. As needed, make changes to your code to generate the bitstream and to verify the functionality of your implementation on hardware.

4.4 – Final Report:

Your final report will be a formal document which demonstrates the work you have done and explains the implementation details of your processor. You would start describing the project from start to the end. It may possibly include the following (and potentially more):

- Your understanding of the project
- Work distribution (who was responsible for what part of the project)
- Description and source code of all components
- Discussions and Simulation results for all components
- Schematic of your Datapath and control unit
- References that helped you in understanding and design
- Conclusion

4.5 – Live Interview and Hardware Demonstration:

At the end of project, you and your teammates will have a live "interview" and demonstration. Each team member will be interviewed and asked questions about the processor your team implemented, and more specifically about their components. Your team will also have to demonstrate the functionality of your processor and prove that it is correctly implemented.

When demonstrating the functionality of your processor, you will need to show both the simulated results of running your processor and a hardware demonstration where you will run your processor directly on an FPGA board.

5 – Processor Specifications

5.1 – Overview

The 16-bit processor is of a simple, single-cycle, non-pipelined design.

- Word Size: 16 bits
- Memory is addressed by: byte (8 bits)
- Non-pipelined

5.2 – Required Components

Your processor will have multiple individual components that will be combined using structural Verilog. There are specific components that you must implement in your design:

Required Components:

- Instruction Memory
- Data Memory
- Register File
- ALU
- Sign Extension
- Control Unit(s)
- Multiplexer(s)
- Program Counter

You may implement additional components as needed by your design.

5.3 – Component Information

- The Register file has 16 registers: $s0 through $s15, all of which are 16bit wide
- The ALU does not need to handle carry and overflow conditions.
- The Instruction Memory has a minimum size of 64 memory locations.
- The Data Memory has a minimum size of 64 memory locations.

5.4 – Instructions

5.4.1 – Instruction Formats

R-type instruction:

opcode				rt/rd				rs				function code			
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0

I-type instruction:

opcode				rt/rd				rs				immediate			
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0

J-type instruction:

opcode				address											
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0

5.4.2 – Mandatory Instructions to Implement

Instruction	Opcode (in binary)	Function Code (in binary)	Type
add	0000	0000	R-type
sub	0000	0001	R-type
sll	0000	0010	R-type
and	0000	0011	R-type
lw	0001	N/A	I-type
sw	0010	N/A	I-type
addi	0011	N/A	I-type
beq	0100	N/A	I-type
bne	0101	N/A	I-type
jmp	0110	N/A	J-type

Note: The 16-bit instruction format allows for many more instructions. You are welcome to add any additional functionality to you processor, but the above instructions MUST be implemented as listed. Only these 10 instructions listed here will be used when evaluating your implementation.

5.4.3 – Detailed Instruction Information

`add`
The add instruction interprets the values of both the registers R[rt/rd] and R[rs] as signed 16-bit integers, computes R[rs]+R[rt/rd] (their sum) and stores the result as a signed 16-bit integer in the register R[rt/rd]. Overflow is undefined behavior (don't worry about it).

`sub`
The sub instruction interprets the values of both R[rt/rd] and R[rs] as signed 16-bit integers, computes R[rs]-R[rt/rd] (their difference), and stores the result as a signed 16-bit integer in the register R[rt/rd]. Underflow is undefined behavior (don't worry about it).

`sll`
The sll instruction interprets the values of both R[rt/rd] and R[rs] as unsigned 16-bit integers and shifts the bits in R[rt/rd] by R[rs] bits with zeros inserted on right to replace the shifted bits. The leftmost shifted bits are discarded. The result is stored in the register R[rt/rd]. Example: If R[rt/rd]=0b00000000 01011010 and R[rs]=4, then executing sll would result in: R[rt/rd]=0b00000000 10100000

`and`
The and instruction takes the values of R[rt/rd] and R[rs], performs bitwise-and, and stores the result as an unsigned 16-bit integer in the register R[rt/rd]. Example: If R[rt/rd]=0b00000000 00001010 and R[rs]=0b00000000 00001100, then executing and would result in: R[rt/rd]=0b00000000 00001000.

`addi`
The addi instruction interprets the value of R[rs] as a 16-bit signed integer and the value of immediate as a 4-bit signed integer, computes their sum, and stores the sum as a signed 16-bit integer in the register R[rt/rd].

`sw`
The sw instruction retrieves the value of R[rs] as an unsigned 16-bit integer and the value immediate as a signed 4-bit integer, then stores the value in R[rt/rd] at the memory address R[rs]+immediate. The value is stored in memory using big-endian.

`lw`
The lw instruction retrieves the value of R[rs] as an unsigned 16-bit integer and the value immediate as a signed 4-bit integer and then loads a word from memory at the address R[rs]+immediate and stores it in the register R[rt/rd]. The value is stored in memory using big-endian.

beq
The beq instruction interprets the values of both R[rt/rd] and R[rs] as signed 16-bit integers and checks if the two values are equivalent. If they are equivalent, the immediate value is interpreted as a signed 4-bit integer. The immediate value is shifted left 1 bit and added to the program counter, where the program counter is the address of the next instruction to be executed. Example: If R[rt/rd]=0b00000000 00001010 and R[rs]=0b00000000 00001010 with the program counter at instruction address 0b00000000 00001110 (the currently executing instructions address + 2) and immediate=0b0010, then executing would result in the program counter branching to instruction address 0b00000000 00010010.

bne
The bne instruction interprets the values of both R[rt/rd] and R[rs] as signed 16-bit integers and checks if the two values are equivalent. If they are not equivalent, the immediate value is interpreted as a signed 4-bit integer. The immediate value is shifted left 1 bit and added to the program counter, where the program counter is the address of the next instruction to be executed. Example: If R[rt/rd]=0b00000000 00000010 and R[rs]=0b00000000 00001010 with the program counter at instruction address 0b00000000 00001110 (the currently executing instructions address + 2) and immediate=0b0010, then executing would result in the program counter branching to instruction address 0b00000000 00010010.

jmp
The jmp instruction interprets the address value as a 12-bit signed integer. The integer is shifted left 1 bit then added with the program counter and the code continues executing from that address. Example: If the program counter is at instruction address 0b00000000 00001110 (the currently executing instructions address + 2) and the address=0b0000 00000010, then executing would result in the program counter branching to instruction address 0b00000000 00010010

6 – Frequently Asked Questions

6.1 – Where do I start?

Start by first designing your datapath for your CPU. Then, start designing major components such as the program counter, ALU, instruction memory, etc. Smaller components such as Sign Extension and Muxes can be implemented as needed. Also start with R-type instructions first, and then later work on implementing I-type instructions. Once the branch I-type instructions are implemented, the J-type instruction can be implemented.

6.2 – How do I design memory (Instruction/Data)?

6.2.1 – Initializing Memory in Verilog

Option 1:

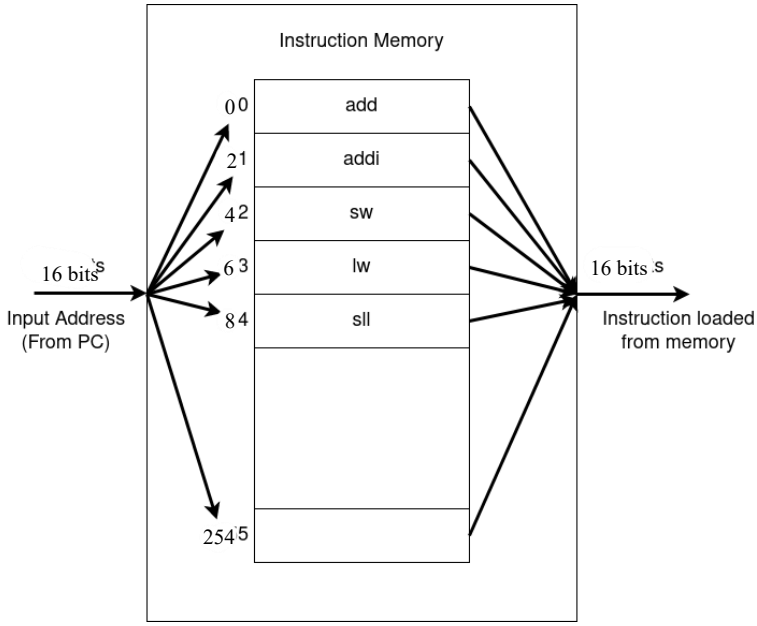
When initializing memory in Verilog, you may use the `readmemb` and `readmemh` commands, both of which read a text file and uses the contents of the text file to initialize an array/memory.

Option 2:

You can simply hardcode initial values for memory by assigning values to memory inside of the initial begin block.

6.2.2 – Instruction Memory Design

In instruction memory, you get an address/index as input and look up the for the opcode of the instruction stored in that address/index of the memory/array (like the figure below):



6.2.3 – Data Memory Design

When designing the Data Memory:

1. Initialize an array with appropriate values.
2. Be sure to consider read and write operations:
    - Read -> Take an address/index of a memory/array as input and retrieve the data in that location and send it as output
    - Write -> Get the data and address/index as input and store it in an array/memory.
3. The control lines should choose between Read and Write depending on the instruction.

6.2.4 – Memory Representation in Verilog

For behavioral Verilog representation, you can consider memory as an array.

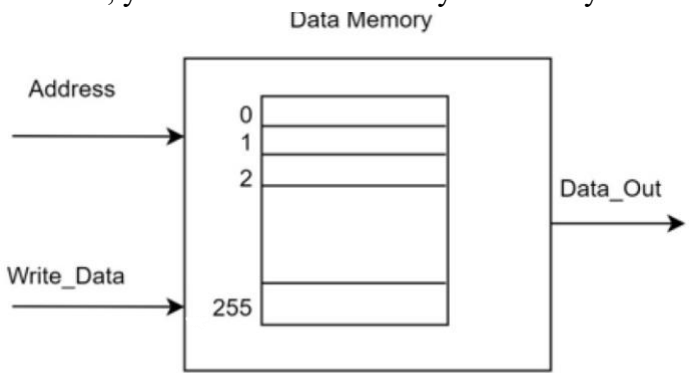


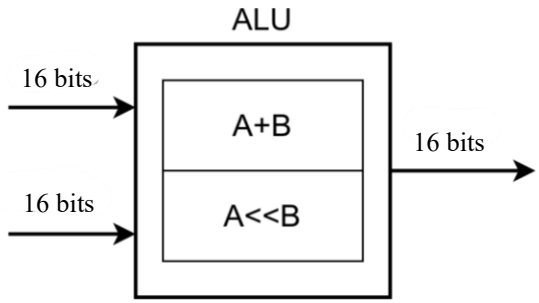
Fig 2. Data Memory Representation

6.2.5 – Why are there minimum and maximum memory sizes?

The 64 byte minimum was arbitrarily chosen. The 65536 byte maximum is a limitation inherent of the processor design itself. Since the processor in this project is a 16-bit processor, memory addresses can only be encoded with 16 bits, allowing for $2^{16} = 65536$ unique addresses. Since the processor is byte-addressed, this means that memory can be *at most* 65536 bytes large. The Basys3 Board has limited memory on it. Including 65536 memory locations for both the Instruction Memory and Data Memory may be too much. We recommend keeping the actual number of locations as low as possible, such as 256 bytes. We will only test up to 64 byte memory size.

6.3 – What type of Adder? Carry or Overflow?

A simple adder as shown below is sufficient. Carry and overflow conditions do NOT need to be considered. (The design below is an example and may not exactly reflect what you need to implement).



6.4 – How exactly do you code/implement an instruction?

The opcode of each instruction is given in the project description. You may start with one type of instruction. If you start with R type (for example, add), store the opcode of an instruction in a memory location/array in your instruction memory. Then design the register file with just support for this instruction, followed by the ALU unit. Then design the control unit such that it can send control signals to these individual components just for the add instruction. Once add is implemented and verified to work correctly, you can update the components you designed to support another instruction (such as addi). Also add additional components, such as mux(es) as needed. (Other instructions may need additional components such as sign extension for I type instructions.)

6.5 – Why do we shift the immediate value of the branch/jump instructions?

Like in MIPS, memory for this processor is byte addressable. Since each instruction is 2 bytes, any branch or jump instructions will only jump an even number of bytes. By shifting the immediate value left 1, we can effectively double the range of the jump since we know the LSB will always need to be a 0. This also keeps the program counter aligned.